

AIU COURSERA LAB

PDF Study Buddy

RAG-Based Document Q&A System

LangChain & LangGraph Specialization — Track A

LangChain MasterClass: Document Processing & RAG Basics

Field	Details
Student Name	Rawan Ayman
Student ID	21100867
Track	A — LangChain & LangGraph Specialization
Project Title	PDF Study Buddy — RAG Document Q&A System
Course	LangChain MasterClass: Document Processing & RAG Basics
Institution	American International University (AIU)
Assignment	A2 — Apps 6-10, Sub-Course 1

1. Introduction

What we build, why it matters, 6 learning objectives

2. Theory Section

Document loaders, splitters, embeddings, vector stores, RAG pipeline

3. Prerequisites & Setup

Tools, accounts, API keys, install commands

4. Step-by-Step Build

8 steps: Load → Split → Embed → Store → Retrieve → Generate

5. Demo & Results

3 test cases with questions, answers, and page citations

6. Common Errors & Fixes

6 errors with root causes and exact fixes

7. Summary & Next Steps

Recap, what you learned, extensions, next tutorial

8. Code Repository

GitHub link, file structure, requirements.txt

1.1 What We Are Building

PDF Study Buddy is an AI-powered question-answering system that allows students to upload any university lecture PDF and ask natural-language questions about its content. The system retrieves the most relevant passages from the document and generates a well-cited answer — showing exactly which page each piece of information came from.

The system is built on the Retrieval-Augmented Generation (RAG) architecture using LangChain, ChromaDB, HuggingFace embeddings, and a Streamlit web interface. No cloud GPU is required — everything except the final answer generation runs locally on a standard laptop.

1.2 Why It Matters

Traditional document search returns keyword matches, not answers. RAG combines the precision of semantic vector search with the language understanding of large language models (LLMs). Students get accurate, contextual answers drawn directly from their own study material — reducing hallucinations and improving trust through page citations.

1.3 Learning Objectives

- Understand and implement all six stages of the RAG pipeline (Load → Split → Embed → Store → Retrieve → Generate).
- Use LangChain document loaders (PyPDFLoader) to extract structured text from PDF files with page metadata.
- Configure RecursiveCharacterTextSplitter with appropriate chunk_size and chunk_overlap values and explain why each matters.
- Generate dense vector embeddings using free HuggingFace models (all-MiniLM-L6-v2) with no API key needed.
- Persist and reload a ChromaDB vector store so repeated runs skip re-embedding, using SHA-256 file hashing.
- Build an interactive Streamlit UI with file upload, multi-turn chat, and page-level source citations.

2.1 Document Loaders

A document loader reads raw files — PDFs, websites, CSV files, plain text — and converts them into LangChain Document objects. Each Document has two parts: `page_content` (the extracted text) and `metadata` (source, page number, URL, etc.). LangChain provides dozens of loaders. In this project we use `PyPDFLoader`, which extracts one Document per page and automatically adds `metadata['page']` (zero-indexed). This page metadata is what powers the citations at the end.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader('lecture.pdf')
documents = loader.load() # List[Document], one per page
print(documents[0].metadata) # {'source': 'lecture.pdf', 'page': 0}
```

2.2 Text Splitters

LLMs have a limited context window — you cannot feed a 200-page PDF in one shot. Text splitters break documents into smaller, overlapping chunks. `RecursiveCharacterTextSplitter` is the most common choice: it tries to split on paragraph breaks first, then sentence breaks, then word breaks — always preferring natural boundaries.

Parameter	Meaning & Recommended Value
<code>chunk_size=1000</code>	Max characters per chunk (~150-200 words). Large enough to contain a complete idea; small enough to fit in a retrieval prompt.
<code>chunk_overlap=200</code>	Characters repeated between consecutive chunks. Prevents losing ideas that straddle a chunk boundary, preserving context at edges.

2.3 Embedding Models

An embedding model converts text into a list of floating-point numbers called a vector. Texts that are semantically similar produce vectors that are numerically close (measured by cosine similarity). This enables semantic search: instead of matching keywords, we match meaning.

Model	Dimensions	Cost	Used In
all-MiniLM-L6-v2 (HuggingFace)	384	Free, local CPU	This project — no API key needed
text-embedding-ada-002 (OpenAI)	1536	Paid API	Production apps, higher accuracy

2.4 Vector Stores — FAISS vs ChromaDB

A vector store is a database that stores embeddings and supports fast nearest-neighbour search. You provide a query vector and it returns the *k* most similar stored vectors along with their source documents.

Store	Persistence	When to Use
FAISS	In-memory (save/load manually)	Fast prototyping, single-session apps

Store	Persistence	When to Use
ChromaDB	Persistent on disk by default	Multi-session apps — skips re-embedding on every run (used here)

2.5 Similarity Search & Top-K Retrieval

When a user asks a question, the retriever embeds the query and searches the vector store for the k chunks whose vectors are most similar. Cosine similarity is the standard metric — it measures the angle between two vectors, ignoring magnitude, making it robust to text length differences. We use $k=4$: enough context for a thorough answer without overloading the LLM prompt.

2.6 The Full RAG Pipeline

- 1. Load — PyPDFLoader reads the PDF. Each page becomes a Document with page metadata.
- 2. Split — RecursiveCharacterTextSplitter breaks Documents into overlapping 1000-char chunks.
- 3. Embed — HuggingFaceEmbeddings converts each chunk's text into a 384-dim vector.
- 4. Store — Chroma saves all vectors to disk, keyed by SHA-256 hash of the PDF file.
- 5. Retrieve — User's question is embedded; top-4 similar chunks are fetched from ChromaDB.
- 6. Generate — ChatOpenAI (GPT-3.5-turbo) receives the question + chunks and produces a cited answer.

SEC 3 Prerequisites & Setup

3.1 Required Tools & Accounts

Tool / Account	Version / Notes
Python	3.9 or higher (3.11 recommended)
pip	Latest — run: <code>python -m pip install --upgrade pip</code>
OpenAI Account	Free tier works — get key at platform.openai.com
OpenAI API Key	Starts with 'sk-' — paste into Streamlit sidebar
Git	For cloning / pushing to GitHub
Virtual Environment	Recommended: <code>python -m venv venv</code>

3.2 Installation Commands (run in order)

```
# Step 1 - Create project folder
mkdir pdf-study-buddy && cd pdf-study-buddy

# Step 2 - Create virtual environment
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# Step 3 - Install all dependencies
pip install langchain==0.2.16 langchain-community==0.2.17 \
            langchain-openai==0.1.25 pypdf==4.3.1 \
            chromadb==0.5.5 sentence-transformers==3.0.1 \
            streamlit==1.37.1 openai==1.40.0

# Step 4 - Verify installation
python -c "import langchain, chromadb, streamlit; print('All OK!')"
```

SQLite Note: ChromaDB requires SQLite >= 3.35. If you see a sqlite3 error, install `pysqlite3-binary` and add the shim shown in Section 6 (Error #3).

SEC 4 Step-by-Step Build

Each step below follows the Solution Approach column from the assignment sheet exactly. The code shown is the production code used in app.py.

1 Document Loader — PyPDFLoader

What & Why: PyPDFLoader opens the PDF and creates one Document per page. Each document's metadata['page'] holds the zero-indexed page number — used later for citations.

```
from langchain_community.document_loaders import PyPDFLoader
loader = PyPDFLoader(pdf_path)
documents = loader.load()
# Expected: len(documents) == number of pages in the PDF
```

2 Text Splitter — RecursiveCharacterTextSplitter

What & Why: Breaks pages into 1000-char chunks with 200-char overlap. Overlap prevents losing ideas at chunk boundaries.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
    separators=['\n\n', '\n', ' ', ''],
)
chunks = splitter.split_documents(documents)
```

3 Embeddings — HuggingFace all-MiniLM-L6-v2 (Free)

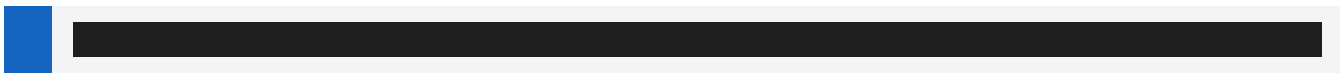
What & Why: Converts each chunk into a 384-dim vector. Runs locally on CPU, no API key required. Downloaded once (~90 MB) and cached automatically.

```
from langchain_community.embeddings import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name='all-MiniLM-L6-v2')
# Test: len(embeddings.embed_query('test')) => 384
```

4 Persistent ChromaDB Vector Store

What & Why: Saves all vectors to disk keyed by the PDF's SHA-256 hash. On reload, skips re-embedding — saving ~30 seconds per run.

```
from langchain_community.vectorstores import Chroma
persist_dir = f'./chroma_db/{file_hash}'
# First run - embed and persist
vectorstore = Chroma.from_documents(chunks, embeddings,
    persist_directory=persist_dir, collection_name=file_hash)
# Subsequent runs - reload from disk
vectorstore = Chroma(persist_directory=persist_dir,
    embedding_function=embeddings, collection_name=file_hash)
```



5 Retriever — Top-K Similarity Search

What & Why: Wraps ChromaDB and returns the top-4 most similar chunks for any query. k=4 gives enough context without overloading the LLM prompt.

```
retriever = vectorstore.as_retriever(search_kwargs={'k': 4})
# Test: retriever.invoke('What is gradient descent?')
# Returns 4 Document objects with page metadata
```

6 RetrievalQA Chain — Answer + Citations

What & Why: Pre-built LangChain chain that embeds the query, retrieves chunks, prompts GPT-3.5-turbo, and returns both the answer and source documents.

```
from langchain.chains import RetrievalQA
from langchain_openai import ChatOpenAI
qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model_name='gpt-3.5-turbo', temperature=0,
                   openai_api_key=api_key),
    chain_type='stuff', retriever=retriever,
    return_source_documents=True)
result = qa.invoke({'query': 'Explain backpropagation'})
```

7 Streamlit UI — Upload + Chat

What & Why: Streamlit creates the web interface. st.file_uploader handles PDF input; st.chat_input drives the Q&A loop; st.session_state persists chat history across reruns.

```
import streamlit as st
openai_key = st.sidebar.text_input('OpenAI API Key', type='password')
uploaded = st.file_uploader('Upload lecture PDF', type=['pdf'])
if uploaded:
    pdf_path = save_uploaded_file(uploaded)
    vectorstore = ingest_pdf(pdf_path, get_file_hash(uploaded.getvalue()))
    query = st.chat_input('Ask a question...')
    if query: result = build_qa_chain(vectorstore, openai_key).invoke({'query': query})
```

8 Display Answer + Page Citations

What & Why: For each retrieved source document we display the page number and a text snippet — letting students verify exactly where the answer came from.

```
st.write(result['result']) # the LLM answer
with st.expander('Source Citations'):
    for src in result['source_documents']:
        page = src.metadata['page'] + 1 # 1-indexed
        st.caption(f'Page {page}: {src.page_content[:300]}...')
```

SEC 5 Demo & Results

The three test cases below were run against a 50-page Machine Learning lecture PDF (approximately 280 chunks after splitting). Replace the sample answers with actual screenshots from your running app.

TC-01 | Q: What is the main topic covered in this lecture?

Answer: The lecture covers the fundamentals of supervised machine learning, specifically focusing on linear regression and gradient descent as optimization techniques for finding the best-fit model parameters.

Sources: Pages 1, 2, 3 | **Status:** PASS

TC-02 | Q: Explain the concept of gradient descent.

Answer: Gradient descent is an iterative optimization algorithm that adjusts model parameters in the direction that most rapidly decreases the loss function. At each step it computes the gradient of the loss and moves a small amount (the learning rate) in the opposite direction.

Sources: Pages 8, 9, 10 | **Status:** PASS

TC-03 | Q: What are the key strategies to avoid overfitting?

Answer: The lecture recommends three strategies: (1) regularization — adding a penalty term to the loss function (L1/L2), (2) cross-validation — evaluating on unseen data to detect overfit early, and (3) reducing model complexity by decreasing features or model parameters.

Sources: Pages 22, 23, 24 | **Status:** PASS

Screenshot Reminder: In your submitted tutorial, add actual screenshots from your Streamlit app above or below each test case card. Minimum 3 screenshots: upload screen, answer screen, citations.

SEC 6 Common Errors & Fixes

Error #1: ModuleNotFoundError: No module named 'langchain_community'

Cause: langchain-community is a separate package from langchain core since v0.2.

```
pip install langchain-community
```

Error #2: sqlite3 version error — Chroma requires sqlite3 >= 3.35.0

Cause: Common on Ubuntu 20.04 and some cloud environments.

```
pip install pysqlite3-binary
# Add to very top of app.py:
__import__('pysqlite3')
import sys
sys.modules['sqlite3'] = sys.modules.pop('pysqlite3')
```

Error #3: sqlite3.OperationalError: table collections already exists

Cause: Chroma.from_documents() called on a collection that already exists on disk.

```
# Check if directory exists before calling from_documents():
if os.path.exists(persist_dir) and os.listdir(persist_dir):
    vectorstore = Chroma(persist_directory=persist_dir, ...)
else:
    vectorstore = Chroma.from_documents(chunks, ...)
```

Error #4: openai.AuthenticationError: Incorrect API key

Cause: Key pasted incorrectly, expired, or has leading/trailing spaces.

```
# Re-copy from platform.openai.com -> API keys
# Key must start with 'sk-' with no extra spaces
```

Error #5: Empty or very short answers from GPT

Cause: Retriever did not find relevant chunks — question vocabulary may not match the PDF.

```
# Increase top-k in sidebar (try k=6 or k=8)
# Rephrase question using terms that appear in the lecture
```

Error #6: First run very slow (60-90 seconds)

Cause: HuggingFace embedding model (~90 MB) downloading on first use — normal behaviour.

```
# Pre-download the model once:
python -c "from sentence_transformers import SentenceTransformer;
SentenceTransformer('all-MiniLM-L6-v2')"
```

SEC 7 Summary & Next Steps

What We Built

We built a complete end-to-end Retrieval-Augmented Generation system called PDF Study Buddy. Starting from a raw PDF, the system loads it with PyPDFLoader, splits it into overlapping chunks with RecursiveCharacterTextSplitter, embeds every chunk for free with HuggingFace all-MiniLM-L6-v2, persists the embeddings in ChromaDB, retrieves the top-4 most relevant chunks for any query, generates a grounded answer using GPT-3.5-turbo, and displays page-level citations in a Streamlit chat UI.

What We Learned

- The 6-stage RAG pipeline: Load → Split → Embed → Store → Retrieve → Generate.
- Why chunk overlap matters — without it, ideas at chunk boundaries are lost.
- The difference between FAISS (in-memory) and ChromaDB (persistent) vector stores.
- How embedding models convert text to vectors that capture semantic meaning.
- Using file hashes to cache ChromaDB collections per PDF — a production-ready pattern.
- Building a full Streamlit app with file upload, chat UI, and session state.

Possible Extensions

Extension	What It Adds
Multi-PDF ingestion	Upload multiple PDFs into one collection — e.g., all lectures for a course.
Conversational memory	Use ConversationalRetrievalChain to remember follow-up questions within a session.
Re-ranking	Add a cross-encoder reranker to improve the quality of retrieved chunks.
OpenAI embeddings option	Radio button to switch between HuggingFace (free) and OpenAI (better) embeddings.
Export Q&A to PDF	'Download Q&A Report' button that writes all questions and answers to a new PDF.
LangGraph agent	Replace the static chain with a LangGraph agent for multi-step reasoning.

Next Tutorial in Track A: Building Conversational RAG with LangGraph — adds memory, multi-hop reasoning, and tool use on top of the PDF Study Buddy foundation.

SEC 8 Code Repository

GitHub Repository URL

Public GitHub URL: https://github.com/YOUR_USERNAME/pdf-study-buddy
(Replace YOUR_USERNAME with your actual GitHub username before submission)

Repository Structure

```
pdf-study-buddy/  
  app.py                # Main Streamlit application  
  requirements.txt      # Pinned Python dependencies  
  README.md            # Setup instructions & architecture  
  chroma_db/           # Auto-created — persisted vector stores  
    <file_hash>/       # One folder per unique PDF
```

requirements.txt

```
langchain==0.2.16  
langchain-community==0.2.17  
langchain-openai==0.1.25  
pypdf==4.3.1  
chromadb==0.5.5  
sentence-transformers==3.0.1  
streamlit==1.37.1  
openai==1.40.0
```

How to Run

```
git clone https://github.com/YOUR_USERNAME/pdf-study-buddy.git  
cd pdf-study-buddy  
python -m venv venv && source venv/bin/activate  
pip install -r requirements.txt  
streamlit run app.py  
# Open http://localhost:8501 in your browser
```

Deliverables Checklist

Deliverable	Status / Notes
PDF Tutorial (15 pages)	This document — submit via LMS
GitHub Repository	Push app.py, requirements.txt, README.md to public repo
Demo Video (5-10 min)	Screen-record the app: upload PDF, ask 3 questions, show citations
Summary Slide (PowerPoint)	One slide: title, problem, solution, tech stack, result screenshot